



Generación y Optimización de Código (80 horas)



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN



➤ Refactorización y mejora de código existente.

- I. *¿Qué es la refactorización?*
- II. *Tipos de refactorización asistida por IA*
- III. *Workflow profesional de refactorización con IA*
- IV. *Mejores prácticas de refactorización con IA*
- V. *Errores comunes al refactorizar con IA*



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

2.1 ¿Qué es la refactorización?

La refactorización es el proceso de mejorar la estructura interna del código sin modificar su comportamiento externo.

Permitiendo transformar código antiguo, complejo o mal estructurado en código limpio, moderno, eficiente y mantenible.

Refactorización y mejora de código existente.



Objetivos principales:

- ☐ Mejorar legibilidad
- ☐ Reducir complejidad
- ☐ Facilitar mantenimiento
- ☐ Mejorar rendimiento
- ☐ Eliminar duplicidades
- ☐ Aplicar patrones de diseño
- ☐ Modernizar código legacy



Refactorización y mejora de código existente.

Beneficios:

- ☐ Menos errores
- ☐ Código más fácil de testear
- ☐ Mejor rendimiento
- ☐ Mayor vida útil del software

2.2 Tipos de refactorización asistida por IA

La IA puede realizar distintos tipos de refactorización según el objetivo.

- ◆ 1. Simplificación
- ◆ 2. Modularización
- ◆ 3. Mejora de nomenclatura
- ◆ 4. Optimización
- ◆ 5. Aplicación de patrones de diseño
- ◆ 6. Modernización de código legacy

Refactorización y mejora de código existente.

■ 2.2 Tipos de refactorización asistida por IA

◆ 1. Simplificación

Reduce complejidad ciclomática.

Ejemplos:

- ☐ Eliminar condicionales anidados
- ☐ Reemplazar bucles complejos
- ☐ Usar early returns

Prompt típico: "Simplifica esta función eliminando anidación"

Refactorización y mejora de código existente.

```
def procesar_pedido(producto, cantidad):  
    if producto:  
        if cantidad > 0:  
            if stock_disponible(producto):  
                if validar_pago():  
                    enviar_pedido()  
                    return "Éxito"  
            else:  
                return "Error pago"  
        else:  
            return "Sin stock"  
    else:  
        return "Cantidad inválida"  
else:  
    return "Producto inválido"
```

◆ Código Original (Anidado)

◆ Código Simplificado

```
def procesar_pedido(producto, cantidad):  
    # Early returns - validaciones primero  
    if not producto:  
        return "Producto inválido"  
  
    if cantidad <= 0:  
        return "Cantidad inválida"  
  
    if not stock_disponible(producto):  
        return "Sin stock"  
  
    if not validar_pago():  
        return "Error pago"  
  
    enviar_pedido()  
    return "Éxito"
```

Refactorización y mejora de código existente.

2.2 Tipos de refactorización asistida por IA

2. Modularización

Divide funciones grandes en piezas pequeñas y reutilizables.

Ejemplos:

- ☐ Extraer funciones
- ☐ Crear servicios independientes
- ☐ Separar responsabilidades

Prompt típico: "Divide esta función de 50 líneas en funciones más pequeñas con responsabilidad única."

```
def procesar_usuario(datos):
    # Validar datos (5 líneas)
    if not datos["nombre"] or len(datos["nombre"]) < 3:
        return "Error: Nombre inválido"
    if datos["edad"] < 18:
        return "Error: Menor de edad"

    # Crear perfil (8 líneas)
    perfil = {
        "nombre": datos["nombre"].title(),
        "edad": datos["edad"],
        "categoria": "Premium" if datos["edad"] > 60 else "Regular",
        "descuento": 0.15 if datos["edad"] > 60 else 0.05
    }

    # Guardar (7 líneas)
    db = abrir_conexion()
    db.insert("usuarios", perfil)
    db.commit()
    db.close()

    return f"Usuario {perfil['nombre']} creado"
```

```
# Módulo 1: Validación
def validar_usuario(datos):
    if not datos["nombre"] or len(datos["nombre"]) < 3:
        return "Error: Nombre inválido"
    if datos["edad"] < 18:
        return "Error: Menor de edad"
    return None

# Módulo 2: Creación
def crear_perfil(datos):
    es_premium = datos["edad"] > 60
    return {
        "nombre": datos["nombre"].title(),
        "edad": datos["edad"],
        "categoria": "Premium" if es_premium else "Regular",
        "descuento": 0.15 if es_premium else 0.05
    }

# Módulo 3: Persistencia
def guardar_usuario(perfil):
    db = abrir_conexion()
    db.insert("usuarios", perfil)
    db.commit()
    db.close()

# Función Principal (Coordinadora)
def procesar_usuario(datos):
    error = validar_usuario(datos)
    if error: return error

    perfil = crear_perfil(datos)
    guardar_usuario(perfil)

    return f"Usuario {perfil['nombre']} creado"
```

◆ ANTES - Función Larga

- ✓ Separación de preocupaciones
- ✓ Código reutilizable
- ✓ Más fácil de mantener
- ✓ Testing unitario posible

◆ DESPUÉS - Modularizado



Refactorización y mejora de código existente.

■ 2.2 Tipos de refactorización asistida por IA

◆ 3. Mejora de nomenclatura

Renombra variables, funciones y clases para que sean más descriptivas.

Ejemplos:

□ $x \rightarrow \text{totalPrice}$

□ $\text{fn1} \rightarrow \text{calculateDiscount}$

Prompt típico: "Renombra variables y funciones siguiendo convenciones de Clean Code."

```
def calc(d, p):
    t = d * p
    if p > 100:
        t = t - 10
    return t
```

```
def prc(lst):
    r = []
    for i in lst:
        if i > 0:
            r.append(i * 2)
    return r
```

```
x = 50
y = 100
res = calc(x, y)
```

◆ ANTES - Nombres Pobres

◆ DESPUÉS - Nombres Claros

✗ calc() → ✓ calcular_total_con_descuento()
 ✗ d, p → ✓ cantidad, precio_unitario
 ✗ t → ✓ total
 ✗ prc() → ✓ procesar_solo_positivos()
 ✗ lst → ✓ numeros
 ✗ i → ✓ numero
 ✗ r → ✓ resultados
 ✗ x, y → ✓ cantidad_productos, precio_unitario
 ✗ res → ✓ total_factura

```
def calcular_total_con_descuento(cantidad, precio_unitario):
    total = cantidad * precio_unitario
    if cantidad > 100:
        total = total - 10 # Descuento por volumen
    return total
```

```
def procesar_solo_positivos(numeros):
    resultados = []
    for numero in numeros:
        if numero > 0:
            resultados.append(numero * 2)
    return resultados
```

```
cantidad_productos = 50
precio_unitario = 100
total_factura = calcular_total_con_descuento(cantidad_productos, precio_unitario)
```

Refactorización y mejora de código existente.

■ 2.2 Tipos de refactorización asistida por IA

◆ 4. Optimización

Mejora rendimiento y uso de memoria.

Ejemplos:

- ☐ Reducir complejidad $O(n^2) \rightarrow O(n \log n)$
- ☐ Usar estructuras más eficientes
- ☐ Evitar cálculos repetidos

Prompt típico:

“Optimiza este algoritmo para mejorar su complejidad temporal.”

ANTES - $O(n^2)$ (Ineficiente)



```
def encontrar_duplicados(lista):
    duplicados = []
    for i in range(len(lista)):
        for j in range(len(lista)):
            if i != j and lista[i] == lista[j]:
                if lista[i] not in duplicados:
                    duplicados.append(lista[i])
    return duplicados
```

Complejidad: $O(n^2) + O(n) \rightarrow O(n^2)$

1. ❌ Dos bucles anidados → ✅ Un solo bucle
2. ❌ Lista para búsquedas → ✅ Set (HashTable)
3. ❌ Buscar en lista $O(n)$ → ✅ Buscar en set $O(1)$

◆ DESPUÉS - $O(n)$ (Optimizado)

```
def encontrar_duplicados(lista):
    duplicados = []
    vistos = set() #  $O(1)$  para buscar

    for numero in lista: #  $O(n)$ 
        if numero in vistos: #  $O(1)$ 
            if numero not in duplicados: #  $O(n)$  pero mejora con set
                duplicados.append(numero)
        else:
            vistos.add(numero) #  $O(1)$ 

    return duplicados
```

Complejidad: $O(n)$ usando set para búsquedas rápidas



Refactorización y mejora de código existente.

■ 2.2 Tipos de refactorización asistida por IA

◆ 5. Aplicación de patrones de diseño

La IA puede sugerir patrones adecuados. Ejemplos:

- ☐ Strategy
- ☐ Factory
- ☐ Singleton (cuando procede)
- ☐ Repository
- ☐ Observer

Prompt típico: "Refactoriza este código usando el patrón Strategy para eliminar condicionales."

◆ ANTES - Condicionales Anidados



```
class CalculadoraPrecios:
    def calcular_precio(self, tipo_cliente, precio_base):
        if tipo_cliente == "regular":
            return precio_base
        elif tipo_cliente == "premium":
            return precio_base * 0.9 # 10% descuento
        elif tipo_cliente == "vip":
            return precio_base * 0.8 # 20% descuento
        elif tipo_cliente == "empresa":
            return precio_base * 0.7 # 30% descuento
        else:
            return precio_base
```

```
# Uso
calc = CalculadoraPrecios()
precio1 = calc.calcular_precio("premium", 100) # 90
precio2 = calc.calcular_precio("vip", 100)      # 80
```

- ✓ Diferentes algoritmos de descuento
- ✓ Múltiples métodos de pago
- ✓ Varios formatos de exportación
- ✓ Distintas estrategias de envío

```
# Factory para crear estrategias
class FabricaDescuentos:
    estrategias = {
        "regular": DescuentoRegular(),
        "premium": DescuentoPremium(),
        "vip": DescuentoVIP(),
        "empresa": DescuentoEmpresa()
    }

    @staticmethod
    def obtener_estrategia(tipo_cliente):
        return FabricaDescuentos.estrategias.get(tipo_cliente, DescuentoRegular())
```

```
# Uso simple
estrategia = FabricaDescuentos.obtener_estrategia("premium")
calc = CalculadoraPrecios(estrategia)
precio = calc.calcular_precio(100) # 90
```

◆ Versión con Factory (más simple)



Refactorización y mejora de código existente.

■ 2.2 Tipos de refactorización asistida por IA

◆ 6. Modernización de código legacy

Convierte código antiguo en versiones modernas.

Ejemplos:

- ☐ jQuery → React
- ☐ Callbacks → async/await
- ☐ Clases → funciones con hooks
- ☐ Python 2 → Python 3
- ☐ Java 8 → Java 17

Prompt típico: "Moderniza este código jQuery a React Hooks."

ANTES - Callbacks (Pyramid of Doom))

```
function procesarUsuario(id, callback) {
  obtenerUsuario(id, function(usuario) {
    obtenerPedidos(usuario.id, function(pedidos) {
      calcularTotal(pedidos, function(total) {
        actualizarPerfil(usuario.id, total, function() {
          callback("Completado");
        });
      });
    });
  });
}
```

- ✓ Código más limpio y legible
- ✓ Mejor manejo de errores (try/catch)
- ✓ Estado manejado centralmente
- ✓ Componentes reutilizables
- ✓ Testing más fácil
- ✓ Mejor performance (Virtual DOM)



```
async function procesarUsuario(id) {
  try {
    const usuario = await obtenerUsuario(id);
    const pedidos = await obtenerPedidos(usuario.id);
    const total = await calcularTotal(pedidos);
    await actualizarPerfil(usuario.id, total);
    return "Completado";
  } catch (error) {
    console.error("Error:", error);
    throw error;
  }
}
```

DESPUÉS - Async/Await (Moderno)



2.3 Workflow profesional de refactorización con IA

- ◆ Paso 1 — Análisis inicial
- ◆ Paso 2 — Plan de refactorización
- ◆ Paso 3 — Ejecución incremental
- ◆ Paso 4 — Validación
- ◆ Paso 5 — Documentación

■ 2.3 Workflow profesional de refactorización con IA

◆ Paso 1 — Análisis inicial

La IA identifica problemas:

- ☐ Code smells
- ☐ Duplicidades
- ☐ Complejidad
- ☐ Falta de modularidad
- ☐ Riesgos de seguridad

Prompt:

"Analiza este código e identifica todos los code smells y áreas de mejora."

◆ Código a Analizar

```
def proc(d):  
    # Calcular total  
    t = 0  
    for i in d:  
        t += i["p"] * i["c"]  
  
    # Aplicar descuento  
    if t > 1000:  
        t = t * 0.9  
    elif t > 500:  
        t = t * 0.95  
  
    # Guardar en BD  
    conn = get_db()  
    cursor = conn.cursor()  
    cursor.execute("INSERT INTO ventas VALUES (?, ?)", (d[0]["id"], t))  
    conn.commit()  
  
    # Enviar email  
    email = d[0]["email"]  
    send_mail(email, f"Total: {t}")
```

- ⚠ Alta cohesión (hace demasiado)
- ⚠ Bajo acoplamiento (todo mezclado)
- ⚠ Falta de validación
- ⚠ Código no reutilizable
- ⚠ Difícil de testear

Code Smells Identificados

✗ Nombres crípticos

- proc → procesar_venta
- d → datos_venta
- t → total
- i → item

✗ Función demasiado larga (4 responsabilidades)

- Cálculo de total
- Aplicación de descuento
- Persistencia en BD
- Envío de email

✗ Hardcoded valores

- 1000, 500, 0.9, 0.95

✗ Manejo inseguro de datos

- d[0]["email"] (asume siempre hay datos)
- No validación de entrada



Refactorización y mejora de código existente.

■ 2.3 Workflow profesional de refactorización con IA

◆ Paso 2 — Plan de refactorización

La IA propone un plan ordenado por impacto.

Prompt:

"Sugiere un plan de refactorización priorizado por impacto y dificultad."

```
def proc(d):  
    # Calcular total  
    t = 0  
    for i in d:  
        t += i["p"] * i["c"]  
  
    # Aplicar descuento  
    if t > 1000:  
        t = t * 0.9  
    elif t > 500:  
        t = t * 0.95  
  
    # Guardar en BD  
    conn = get_db()  
    cursor = conn.cursor()  
    cursor.execute("INSERT INTO ventas VALUES (?, ?)", (d[0]["id"], t))  
    conn.commit()  
  
    # Enviar email  
    email = d[0]["email"]  
    send_mail(email, f"Total: {t}")  
  
    return t
```

◆ Plan de Refactorización Priorizado

● FASE 1: RIESGO BAJO - ALTO IMPACTO (Hoy)



```
# Paso 1.1: Renombrar variables crípticas (15 min)
def procesar_venta(datos_venta):
    total = 0
    for item in datos_venta:
        total += item["precio"] * item["cantidad"]

# Resto del código igual...
```

● FASE 2: RIESGO MEDIO - MEJORA LEGIBILIDAD (Esta semana)

```
# Paso 2.1: Extraer valores mágicos a constantes
DESCUENTO_GRANDE = 0.9 # 10% descuento
DESCUENTO_PEQUEÑO = 0.95 # 5% descuento
UMBRAL_GRANDE = 1000
UMBRAL_PEQUEÑO = 500

# Paso 2.2: Crear función para calcular total
def calcular_total(items):
    total = 0
    for item in items:
        total += item["precio"] * item["cantidad"]
    return total
```

◆ Plan de Refactorización Priorizado



```
# Paso 3.1: Dividir responsabilidades
def procesar_venta(datos_venta):
    # 1. Validar entrada
    if not datos_venta:
        raise ValueError("Datos de venta vacíos")

    # 2. Calcular
    total = calcular_total(datos_venta)
    total_con_descuento = aplicar_descuento(total)

    # 3. Persistir
    guardar_venta(datos_venta[0]["id"], total_con_descuento)

    # 4. Notificar
    notificar_cliente(datos_venta[0]["email"], total_con_descuento)

    return total_con_descuento
```

◆ Plan de Refactorización Priorizado

- | | |
|---|--|
| <p>✓ ANTES:</p> <ul style="list-style-type: none">• 1 función (20 líneas)• 4 responsabilidades• 0 validaciones• Difícil de testear• Código acoplado | <p>✓ DESPUÉS:</p> <ul style="list-style-type: none">• 4-5 funciones (3-8 líneas c/u)• 1 responsabilidad por función• Validaciones en entrada/salida• Fácil testing unitario• Bajo acoplamiento |
|---|--|

Refactorización y mejora de código existente.

2.3 Workflow profesional de refactorización con IA

◆ Paso 3 — Ejecución incremental

Refactorizar por partes, no todo a la vez.

Buenas prácticas:

- ☐ Cambios pequeños
- ☐ Commits frecuentes
- ☐ Tests después de cada cambio

Refactorización y mejora de código existente.



■ 2.3 Workflow profesional de refactorización con IA

◆ Paso 4 — Validación

La IA genera tests para asegurar que el comportamiento no cambió.

Prompt:

"Genera tests unitarios para verificar que el comportamiento no cambió tras la refactorización."

◆ Paso 5 — Documentación

La IA actualiza documentación y comentarios.

Prompt:

"Actualiza la documentación explicando los cambios realizados."



```
def calcular_precio(cantidad, precio):  
    total = cantidad * precio  
    if cantidad > 10:  
        total = total * 0.9 # 10% descuento  
    return total
```

```
import pytest
```

```
def test_precio_normal():  
    """Test: Cantidad pequeña sin descuento"""  
    assert calcular_precio(5, 100) == 500 # 5 * 100  
  
def test_precio_con_descuento():  
    """Test: Cantidad grande con descuento"""  
    assert calcular_precio(20, 100) == 1800 # 2000 * 0.9  
  
def test_precio_borde():  
    """Test: Límite exacto del descuento"""  
    assert calcular_precio(10, 100) == 1000 # Sin descuento  
    assert calcular_precio(11, 100) == 990 # Con descuento (1100 * 0.9)  
  
def test_precio_cero():  
    """Test: Caso extremo - cantidad cero"""  
    assert calcular_precio(0, 100) == 0
```

◆ Paso 4: Validación (Tests Unitarios)



```
def calcular_precio(cantidad, precio):  
    total = cantidad * precio  
    if cantidad > 10:  
        total = total * 0.9 # 10% descuento  
    return total
```

◆ Tests Después de Refactorización

Código Refactorizado

```
def calcular_precio(cantidad, precio):  
    CANTIDAD_DESCUENTO = 10  
    PORCENTAJE_DESCUENTO = 0.9  
  
    total = cantidad * precio  
    if cantidad > CANTIDAD_DESCUENTO:  
        total = total * PORCENTAJE_DESCUENTO  
    return total
```

Mismos tests deben seguir pasando

```
def test_refactorizacion():  
    """Todos los tests anteriores deben seguir funcionando"""  
    assert calcular_precio(5, 100) == 500 # ✓  
    assert calcular_precio(20, 100) == 1800 # ✓  
    assert calcular_precio(10, 100) == 1000 # ✓  
    assert calcular_precio(11, 100) == 990 # ✓  
    assert calcular_precio(0, 100) == 0 # ✓
```

Documentación ANTES (Mala)

```
def calc(x, y):  
    # Calcula algo  
    z = x * y  
    if x > 10:  
        z = z * 0.9  
    return z
```

REFACTORIZACIÓN COMPLETA:

1. CÓDIGO MEJORADO ✓

2. TESTS QUE PASAN ✓

3. DOCUMENTACIÓN ACTUALIZADA ✓

ANTES: calc(x, y) → misterioso

AHORA: calcular_precio_con_descuento() → claro



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN

```
def calcular_precio_con_descuento(cantidad, precio_unitario):
```

```
    """
```

```
    Calcula el precio total aplicando descuento por volumen.
```

```
    Args:
```

```
        cantidad (int): Número de unidades a comprar
```

```
        precio_unitario (float): Precio por unidad
```

```
    Returns:
```

```
        float: Precio total con descuento aplicado si corresponde
```

```
    Ejemplos:
```

```
        >>> calcular_precio_con_descuento(5, 100)
```

```
        500.0
```

```
        >>> calcular_precio_con_descuento(20, 100)
```

```
        1800.0
```

```
    Reglas de negocio:
```

```
        - Descuento del 10% aplica para compras mayores a 10 unidades
```

```
        - No hay descuento para 10 unidades o menos
```

```
    """
```

```
    UMBRAL_DESCUENTO = 10
```

```
    PORCENTAJE_DESCUENTO = 0.9 # 10% de descuento
```

```
    precio_total = cantidad * precio_unitario
```

```
    if cantidad > UMBRAL_DESCUENTO:
```

```
        precio_total = precio_total * PORCENTAJE_DESCUENTO
```

```
    return precio_total
```

Documentación DESPUÉS (Clara)

Refactorización y mejora de código existente.

2.4 Ejemplos prácticos de refactorización

◆ Ejemplo 1 — Simplificación de condicionales

Código original: condicionales anidados.

Código refactorizado: early returns + constantes semánticas.

◆ Ejemplo 2 — Optimización $O(n^2) \rightarrow O(n)$

Código original: doble bucle.

Código refactorizado: uso de sets.

◆ Ejemplo 3 — Aplicación de patrón Factory

Código original: múltiples condicionales para crear objetos.

Código refactorizado: Factory centralizado.

2.5 Mejores prácticas de refactorización con IA

- ☐ Tener tests antes de refactorizar
- ☐ Refactorizar en incrementos
- ☐ Validar cada cambio
- ☐ Revisar código generado por IA
- ☐ Medir métricas antes/después
- ☐ Documentar decisiones

■ 2.6 Errores comunes al refactorizar con IA

- ☐ Aceptar cambios sin entenderlos
- ☐ Refactorizar demasiado de golpe
- ☐ No tener tests previos
- ☐ No validar rendimiento
- ☐ No revisar seguridad



Gracias.



MINISTERIO
DE EDUCACIÓN, FORMACIÓN PROFESIONAL
Y DEPORTES

SECRETARÍA GENERAL DE FORMACIÓN PROFESIONAL

DIRECCIÓN GENERAL DE PLANIFICACIÓN, INNOVACIÓN Y GESTIÓN
DE LA FORMACIÓN PROFESIONAL

SUBDIRECCIÓN GENERAL DE PROGRAMAS Y GESTIÓN



